



LIONSDEV

Curso de Programação

MÓDULO 11

Frontend NoCode com FlutterFlow



Professor Nicolas Cardoso Motta

Revisando o Módulo 10



DNS e HTTPS

O nome vira um endereço IP; o HTTPS protege o que vai e o que volta.



API preparada

Porta de `process.env.PORT`, script `start` e uma rota `/health`.



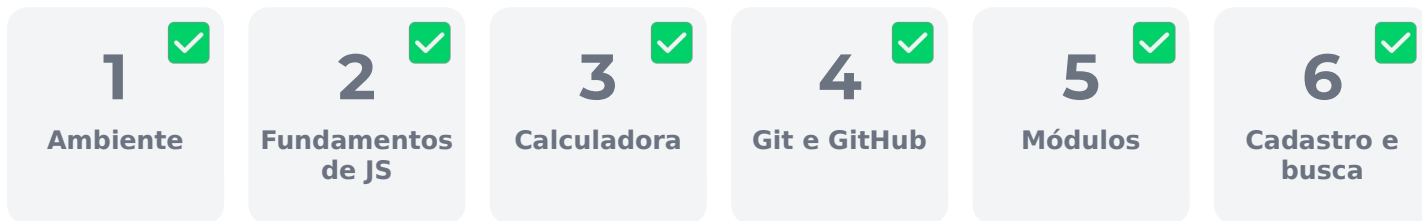
Render

Deploy pelo `render.yaml` ou manual; variáveis no painel, nunca no GitHub.

Pergunta para abrir: por que a primeira requisição à API no plano free às vezes demora quase um minuto?

Mapa do curso

BASE: JAVASCRIPT, GIT E ORGANIZAÇÃO



BACKEND



FRONTEND E PROJETO



Você está aqui: **Módulo 11**, Frontend NoCode com FlutterFlow

O projeto flashcards



NO MÓDULO 11

O flashcards ganha tela: login, lista de baralhos e cartões no FlutterFlow, consumindo a API publicada no Módulo 10. O projeto fica completo.



Objetivo

- Entender o que é NoCode e quando faz sentido usar.
- Conhecer o FlutterFlow e os principais widgets.
- Criar telas com estado, formulários, validação e navegação.
- Consumir a API publicada no Render: listar, enviar dados e fazer login com JWT.
- Tratar os estados de carregando, lista vazia e erro da API.
- Testar e publicar o app.

Nos módulos anteriores construímos a API. Agora vamos criar a **interface que consome essa API, sem escrever o código do frontend.**



O que é NoCode?

NoCode: vantagens e limites

Criar aplicações por uma interface visual, arrastando e configurando componentes prontos.



Vantagens

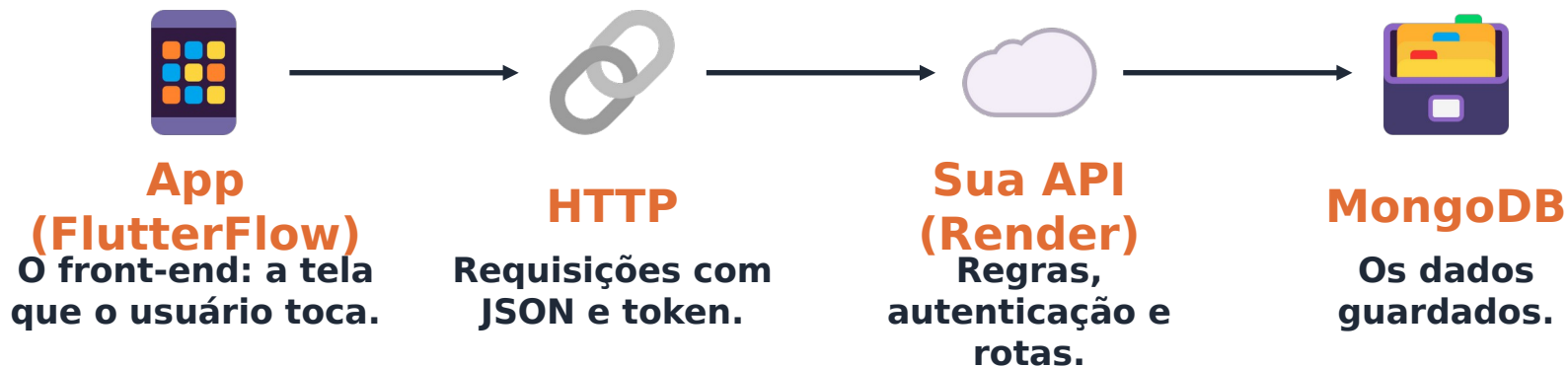
- Cria soluções sem dominar programação de interfaces.
- Muito mais rápido para prototipar.
- Visual: você vê a tela enquanto monta.



Limites

- Menos flexível que o código puro.
- Dependência da plataforma e do plano.
- Lógicas complexas ficam difíceis.

Onde o FlutterFlow se encaixa



O FlutterFlow não substitui o backend: ele conversa com a API que você construiu.



Conhecendo o FlutterFlow

O que é o FlutterFlow?



Apps de verdade

Plataforma NoCode/LowCode para apps móveis e web.



Baseado em Flutter

Gera código Flutter real, que pode ser exportado.



Integra com APIs

Consome qualquer API REST, como a sua no Render.



FlutterFlow

flutterflow.io

A interface do editor



Widget Tree

A árvore com todos os elementos da tela, um dentro do outro.



Canvas

A pré-visualização da tela, no centro.



Painel de propriedades

Tamanho, cor, texto e as ações do widget selecionado.



Widget Palette

Os componentes que você arrasta para a tela.



Widgets

Os blocos de construção da tela

Column x Row



Column

Empilha os widgets na vertical, um embaixo do outro.



Row

Alinha os widgets na horizontal, lado a lado.

Widget é todo elemento visual: texto, botão, imagem, lista. Toda tela é uma árvore de widgets.

Stack, Container e AppBar



Stack

Sobrepõe widgets em camadas: um texto em cima de uma imagem.



Container

Uma caixa com tamanho, cor, borda e espaçamento.



AppBar

A barra fixa no topo, com o título e as ações da página.



Estado e formulários

Variáveis e estado



Estado

A informação que o app guarda enquanto roda: o que muda na tela quando o usuário interage.



Page State

Vale só dentro de uma página, como um filtro de busca.



App State

Vale no app inteiro, como o token de login.

Formulários e validação



TextField

- Captura o que o usuário digita.
- Cada campo vira uma variável: `emailInput`, `senhaInput`.



Validação

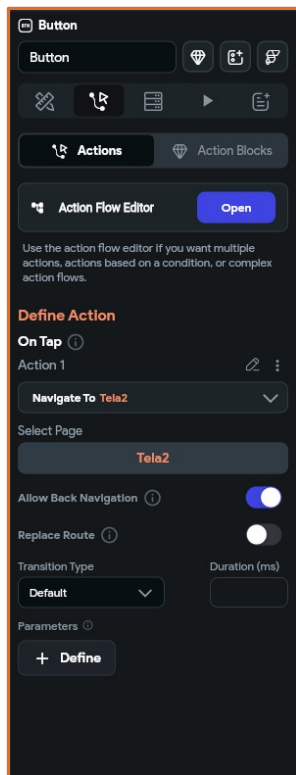
- E-mail obrigatório e no formato certo.
- Senha com um tamanho mínimo.
- Só dispara o POST se estiver tudo válido.

Nunca confie no que o usuário digita: valide no app e na API. A validação do app melhora a experiência; a da API garante a segurança.



Navegação entre telas

Chamando uma tela em um botão



- **Selecione o botão e abra a aba Actions.**
- **Adicione a ação Navigate To e escolha a página de destino.**
- **Rode em modo de teste e clique para conferir.**

O Action Flow Editor

Um toque pode disparar vários passos em sequência:



1. Validar o formulário

Os campos estão preenchidos e no formato certo?



2. Chamar a API

Dispara a API Call com os dados digitados.



3. Decidir

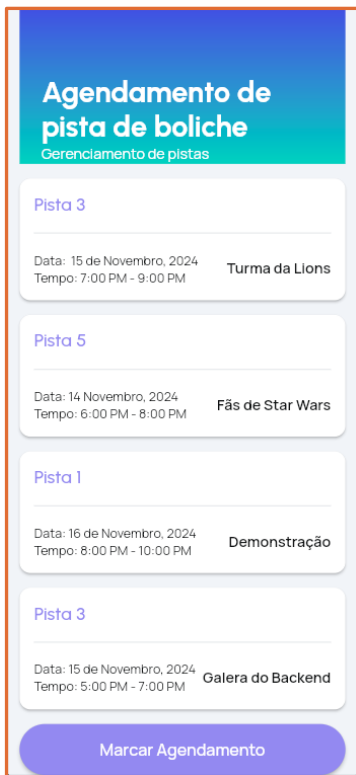
Deu certo? Mostra sucesso. Deu erro? Mostra a mensagem.



4. Navegar

Leva o usuário para a próxima tela.

Atividade: reproduza a tela



- Use só os widgets que vimos: **AppBar, Column, Row, Container e Text.**
- Comece pela **Widget Tree**: primeiro a estrutura, depois as cores e os textos.
- Mais tarde, esta tela vai receber os dados da sua **API**.



Integração com a API

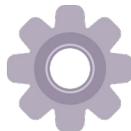
O fluxo da integração

Do endpoint à tela, sempre na mesma ordem:



1. Criar

Uma API Call
para o
endpoint.



2. **Configurar**
Método, URL,
headers e
body.



3. Testar

Ver o JSON
que volta.



4. Mapear

Descrever os
campos num
Data Type.



5. Exibir

Ligar os dados
aos widgets.

Passos 1 e 2: criar e configurar a API Call

Define API Call
Provide the name and configuration for this API call.

Call Definition | Response & Test

API Call Name
GET agendamentos

Method Type: GET | API URL: https://lions-academy-avaliacoes.onrender.com/bookings

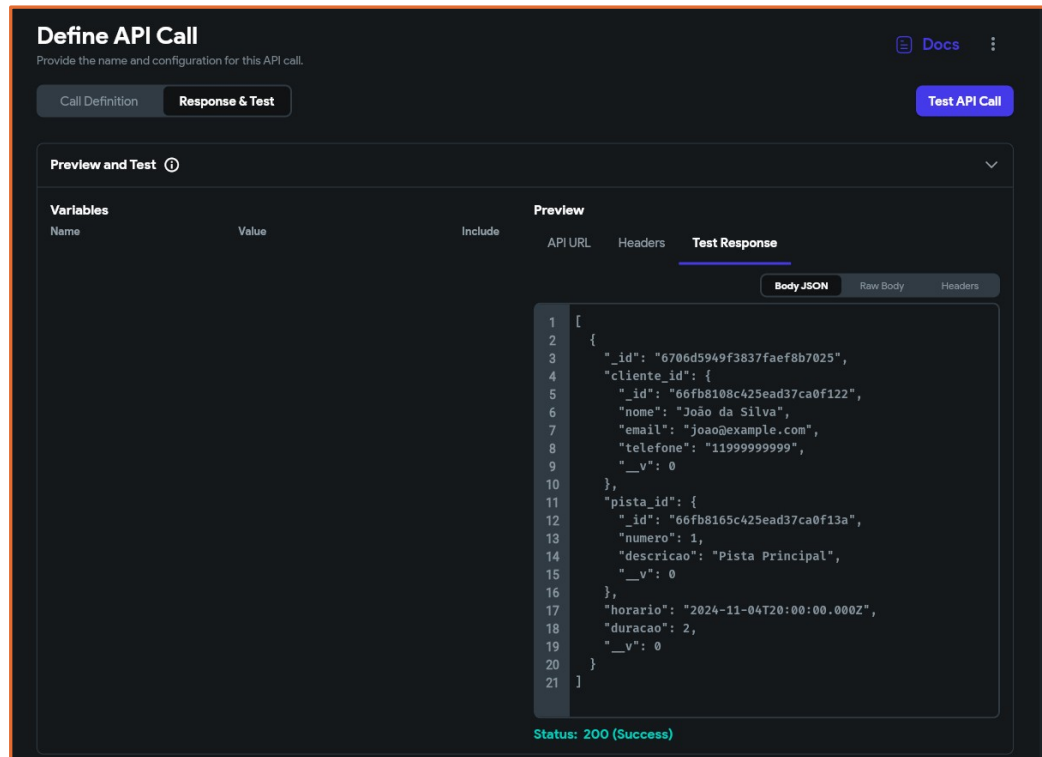
Headers | Query Parameters | Variables | Advanced Settings

Headers
+ Add Header

Delete | Save

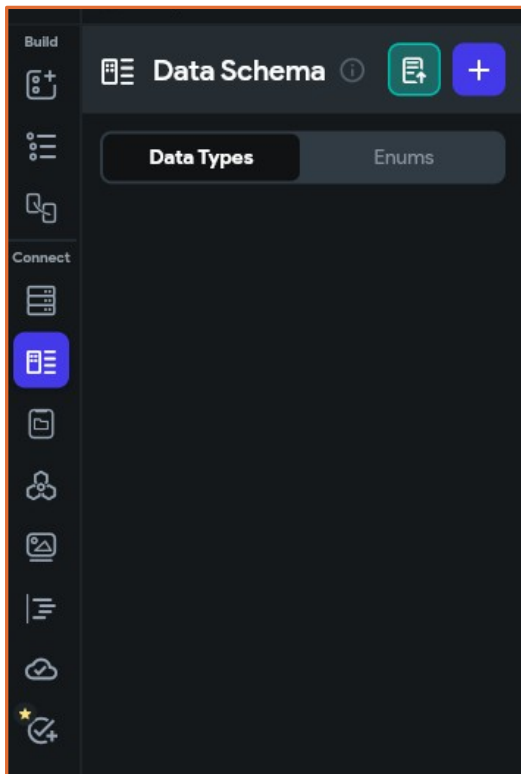
- No menu **API Calls**, crie uma chamada com um nome claro, como **GET agendamentos**.
- Escolha o método e cole a **URL do Render**: **https://sua-api.onrender.com/...**
- Rotas protegidas levam o header **Authorization: Bearer [token]**.

Passo 3: testar a rota



- Na aba Response & Test, clique em Test API Call.
- Você vê exatamente o JSON que a API devolve, antes de ligar na tela.
- Deu erro aqui? O problema está na chamada ou na API, não na tela.

Passo 4: mapear a resposta



- Crie um Data Type com os campos que a API devolve.
- Use o nome exato de cada campo do JSON e o tipo certo: String, Integer, Boolean.
- Um nome diferente do JSON faz o campo aparecer vazio.

Passo 5: exibir na tela



Executar ao abrir a página

Um Backend Query faz o request assim que a tela abre.



Gerar itens dinâmicos

Um item de exemplo, repetido para cada elemento da lista da API.



Ligar os campos

Cada Text mostra um campo do item: nome, data, serviço.

Enviando dados e fazendo login



POST em um botão

- O formulário monta o body com as variáveis dos campos.
- O botão dispara a API Call; PUT e DELETE seguem o mesmo caminho.



Login com JWT

- **POST** /api/auth/login devolve o token.
- Guarde o token no App State (persistido).
- Envie no header das rotas protegidas; no logout, limpe.



Cuidado: localhost não existe no celular

No app, **localhost** é o **próprio celular**, não o seu computador. A chamada simplesmente falha.

- Use sempre a URL pública do Render, em https.
- Lembre do cold start do plano free: a primeira chamada pode demorar. Mostre um indicador de carregamento.



Cuidado: a API precisa aceitar o navegador

No modo de teste web, o navegador bloqueia chamadas para outro domínio se a API não liberar o **CORS**.

- O erro aparece no console do navegador como "blocked by CORS policy".
- Libere na API com o pacote **cors**, antes das rotas.

```
import cors from 'cors'  
app.use(cors()) // npm install cors
```

Publicar e boas práticas



Testar

Test Mode e Run Mode rodam o app no navegador ou no celular.



Publicar

Web com um clique; lojas de app exigem conta de desenvolvedor.



Segurança

Nunca deixe um token fixo no app: use App State e headers.



Nomes claros

API Calls, variáveis e páginas com nomes que dizem o que fazem.



Carregando, vazio e erro

A tela também precisa funcionar quando a API demora, não tem dados ou falha

Os quatro estados de uma tela com API



Carregando

A resposta ainda não chegou. O usuário vê um indicador, não uma tela parada.



Vazio

Deu certo, mas a lista veio []. Uma mensagem e um convite: "cadastre o primeiro".



Erro

A API respondeu 4xx ou 5xx, ou nem respondeu. A mensagem da API aparece na tela.



Sucesso

Os dados na tela: o único estado que a gente costuma testar.

Uma tela que só pensa no sucesso fica em branco, ou travada, nos outros três casos.

Carregando: o Loading Widget



Onde fica

Na Backend Query da ListView (ou da página), abra o Backend Query Loading Widget.



O que mostrar

Uma imagem ou um Component seu: um indicador de progresso com o texto "Carregando...".



Por que importa

O cold start do Render (Módulo 10): a primeira chamada pode demorar, e tela parada parece app quebrado.

Lista vazia: o Empty List Widget



Onde fica

Selecione a ListView que tem a Backend Query e ligue o Show Empty List Widget.



O que mostrar

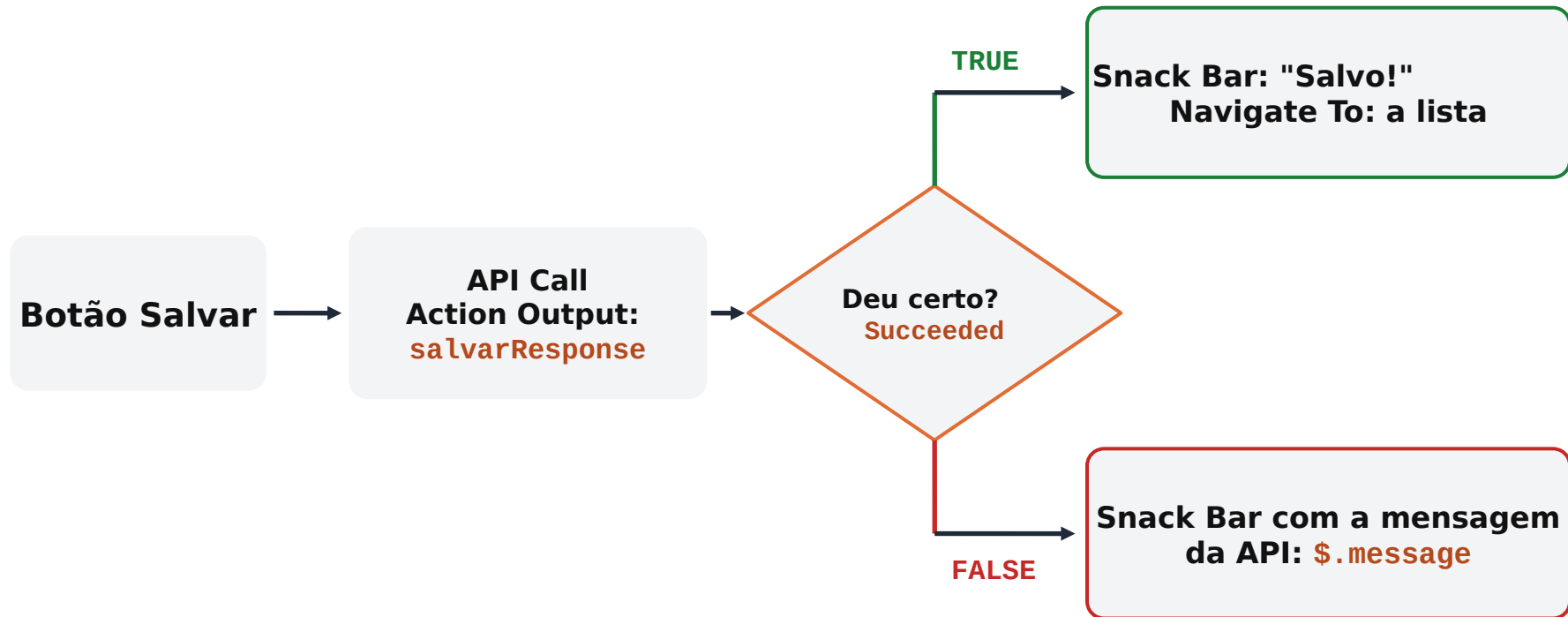
Um Component com mensagem e ação: "Nenhum agendamento ainda" e um botão "Criar o primeiro".



Teste de verdade

Crie um usuário novo, sem dados, e abra a tela: é assim que o cliente vê o app no primeiro dia.

Erro na tela: conferindo a resposta



Depois da API Call: Add Conditional → Action Outputs →
salvarResponse → Succeeded.

Qual mensagem mostrar?

```
POST /api/auth/login
{ "email": "ana@email.com", "senha": "errada" }

→ 401 Unauthorized
{ "message": "Email ou senha incorretos." }
```

\$.message → na Action
Output: **JSON Body** com o
JSON Path **\$.message**.

Status Code → vem na
mesma variável: dá para
decidir o que fazer por
código.

Módulo 9 → o middleware de
erro padronizou: todo erro da
API tem o campo **message**.



Cuidado: 401 no meio do uso

Quando o token vence, **toda** rota protegida responde 401. O app precisa perceber e levar o usuário para o login.

- No caminho FALSE, confira o Status Code: se for 401, limpe o token do App State e navegue para a tela de Login.
- Outros erros (400, 404, 409): mostre a **message** da API e deixe o usuário tentar de novo.
- Na própria tela de login, 401 é só senha errada: ali, basta mostrar a mensagem.

Evitando o clique duplo

- 1 **Crie um Page State**
salvando, do tipo Boolean, começando em false.
- 2 **Antes da API Call**
No Action Flow do botão: Update Page State → **salvando = true**.
- 3 **Depois da resposta**
Nos dois caminhos, TRUE e FALSE: salvando = false.
- 4 **Na tela**
Com Conditional Visibility, esconda o botão e mostre um indicador de progresso enquanto salvando for true.

Sem isso, dois cliques rápidos viram dois cadastros iguais.



Publicando o app

Do editor para a internet

Três caminhos para publicar



Web Publishing

- Direto do FlutterFlow, num subdomínio gratuito.
- Domínio próprio: a partir do plano Basic.



Baixar o código

- Plano Basic ou superior.
- Você hospeda onde quiser, como no Render.



Lojas de apps

- Plano Basic ou superior.
- Conta de desenvolvedor: Google Play e Apple App Store cobram à parte.

No plano gratuito dá para aprender, testar e publicar na web. Baixar o código exige plano pago: confira os limites atuais em flutterflow.io/pricing.

Caminho 1: Web Publishing

- 1 Abra a publicação web**
No projeto, vá até as opções de Web Publishing.
- 2 Escolha o subdomínio**
Algo como `meu-app.flutterflow.app`.
- 3 Publique**
Em poucos minutos o app está no ar, com HTTPS.
- 4 Teste com a API real**
As API Calls precisam apontar para a URL do Render, não para localhost.

Caminho 2: baixando o código



Download Code

No plano Basic, o FlutterFlow gera um projeto Flutter completo, em Dart, para baixar.



GitHub

No plano Growth, o código pode ir direto para um repositório.



O que vem no projeto

`pubspec.yaml` (o "package.json" do Flutter), a pasta `lib/` com as telas e a pasta `web/`.

Rodando o código no computador

```
$ flutter doctor           # confere a instalação
$ cd meu_app
$ flutter pub get          # instala as dependências
$ flutter run -d chrome    # roda no navegador
```

Instale o Flutter SDK em flutter.dev. O flutter pub get é o npm install do Flutter.

Gerando a versão web

```
$ flutter build web
```

```
build/web/  
├── index.html  
├── main.dart.js  
├── assets/  
└── ...
```

flutter build web → compila o app para HTML, CSS e JavaScript.

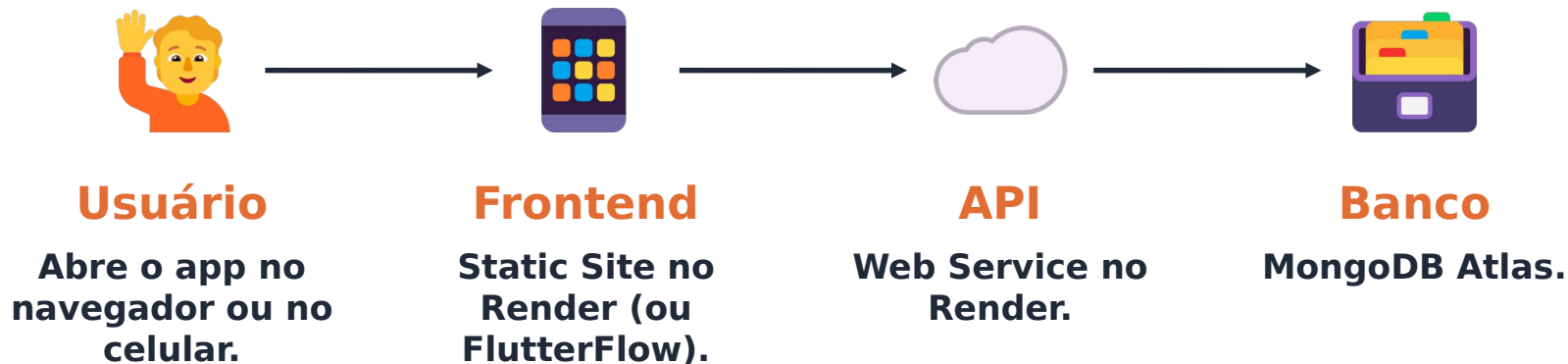
build/web → um site estático: pode ser hospedado em qualquer lugar.

Render → essa pasta vira um Static Site no Render, ao lado da sua API.

Deploy do frontend no Render

- 1 New > Static Site**
Conecte o repositório com a pasta `build/web`.
- 2 Publish Directory: `build/web`**
Como a pasta já vem pronta, não precisa de build (se o campo for obrigatório, use um comando simples, como `echo ok`).
- 3 Rewrite `/*` → `/index.html`**
Em Redirects/Rewrites: sem isso, recarregar uma tela interna dá 404.
- 4 Deploy**
O app ganha uma URL `.onrender.com` própria, com HTTPS.

A arquitetura completa



Tudo o que foi construído no curso, conectado: o projeto final do Módulo 12 segue exatamente essa arquitetura.



Cuidado: frontend e API são dois serviços

Front e API em domínios diferentes: a API precisa **liberar a origem do front, senão o navegador bloqueia.**

- Libere a URL do front no CORS da API.
- No app, a URL base das API Calls é a da API no Render, sempre em https.
- No plano free os dois dormem: a primeira abertura pode demorar.

```
app.use(cors({ origin: 'https://meu-app.onrender.com' })))
```

Atividade Prática

FlutterFlow

1. Monte a tela de listagem da atividade anterior.
2. Crie a API Call para listar os itens da sua API publicada no Render.
3. Teste a chamada, crie o Data Type e gere os itens dinâmicos.
4. Bônus: uma tela de cadastro com POST e uma tela de login com JWT.



Dica: teste cada API Call na aba Response & Test antes de ligar na tela.



Desafio: trate os quatro estados: carregando, lista vazia, erro com a message da API e sucesso.

Listas de exercícios



FlutterFlow: listagem com API

Consumir uma API real e listar os dados.



Frontend com IA

Gerar e adaptar um frontend com ajuda de IA.

No repositório: `modulo11/lista_de_exercicios/`

Desafio: telas do flashcards

Fio condutor

Crie no FlutterFlow as telas do flashcards, consumindo a API publicada no Módulo 10.

- **Login:** chame POST /api/auth/login e guarde o token num App State.
- **Baralhos:** API Call GET /api/baralhos com o header Authorization: Bearer [token].
- **Mostre os baralhos numa lista e trate os quatro estados:** carregando, vazio, erro e sucesso.
- **Ao tocar num baralho, abra a tela com os flashcards dele.**



Cuidado: use a URL do Render, não localhost: o celular não enxerga o seu computador.



Desafio: um botão para criar baralho, com Snack Bar de sucesso e de erro.

Material do módulo



Aponte a câmera do celular

`github.com/nicolassmotta/curso-lions-dev/
tree/main/modulo11`



conteudo/
teoria comentada e o cheat sheet do módulo



lista_de_exercicios/
listas para praticar em casa



Fim!

Agora você tem uma aplicação completa: frontend, API e banco de dados.